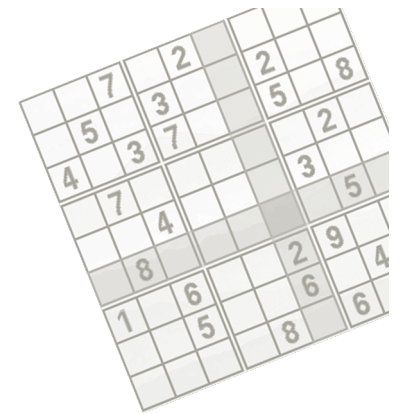




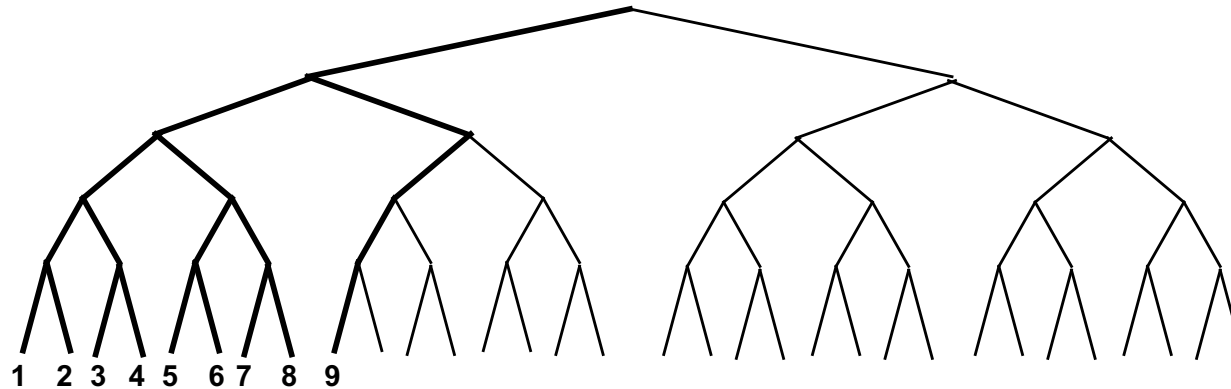
Constraint Programming

Roman Barták

Department of Theoretical Computer Science and Mathematical Logic

Search and Optimization Techniques

Let us go back to foundations: DFS = Depth First Search



Observation:

Real-life problems have huge states spaces that cannot be fully explored.

We can explore just part of the state space!

↳ **incomplete tree search**

Do not explore the state space completely.

- Do not guarantee proving that there is no solution and they do not guarantee finding a solution (completeness)
 - sometimes completeness can be guaranteed with time-complexity trade-off

For many problems, incomplete techniques **find solutions faster**.

Frequently **based on a complete search algorithm** such as DFS.

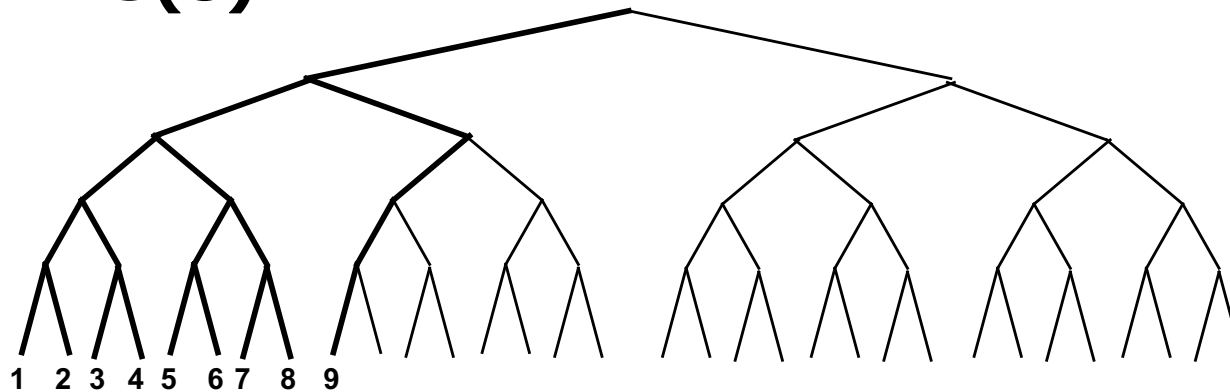
- **Cutoff**
 - after exploiting allocated resources (time, backtracks, credit, ...)
 - may be global (for the whole search tree) or local (for a given sub-tree or a search node)
- **Restart** → *preserves completeness when using cut off*
 - with different parameters (for example with more resources)
 - learning can be used before the next iteration

Limited number of backtracks (cutoff)

- backtracking is counted from the point with other alternatives
- “limited number of leafs”

After failure, **increase the limit by one** (restart).

Example: BBS(8)



Implementation:

- count the number of backtracks (failures)
- stop after exceeding the limit

*↗ cutoff
limit
 $O(k)$*

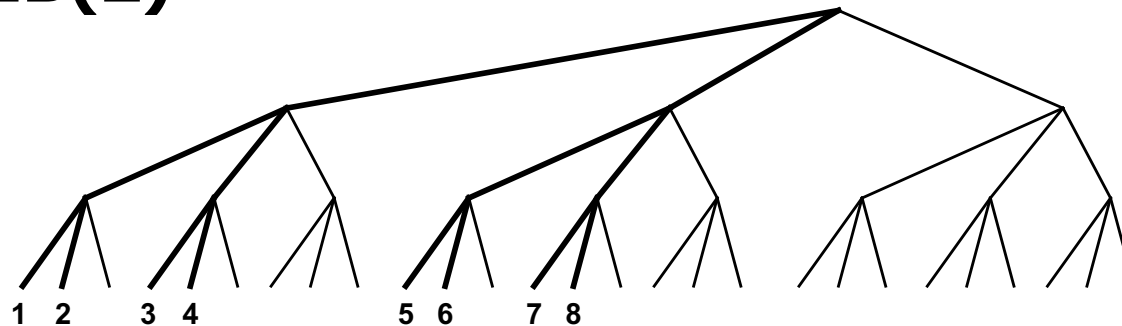
Limited number of alternatives (width) for each node (cutoff).

- try a given number of alternatives for each node
- Beware, this is still exponential!

$$O(\exp^k)$$

After failure, **increase the width by one** (restart).

Example: IB(2)



Implementation:

- restrict the number of tries in nodes (inner for-loop)

↳ improving diversity toho, jakou část stromu zkoumám

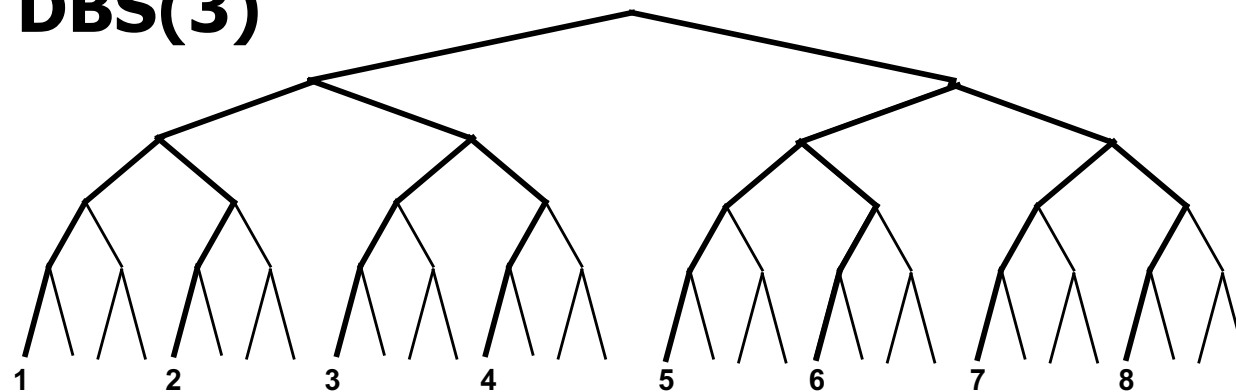
Limited depth for tree search (cutoff).

→ rozhodnutí jen h vms
↳ limit
pro hloubku

- till given depth, all alternatives are tried
- after exceeding the depth, another incomplete search can be used

After failure, **increase the depth by one** (restart).

Example: DBS(3)



konstanta
+ O(jine'ho inc.s.)

Implementation:

- keep the number of instantiated variables
- if the number is larger than a given limit, try just one alternative – BBS(0)

dobrý, když netuším, kde ve stromu to řešení může být

Limited credit (the number of backtracks) for search (cutoff).

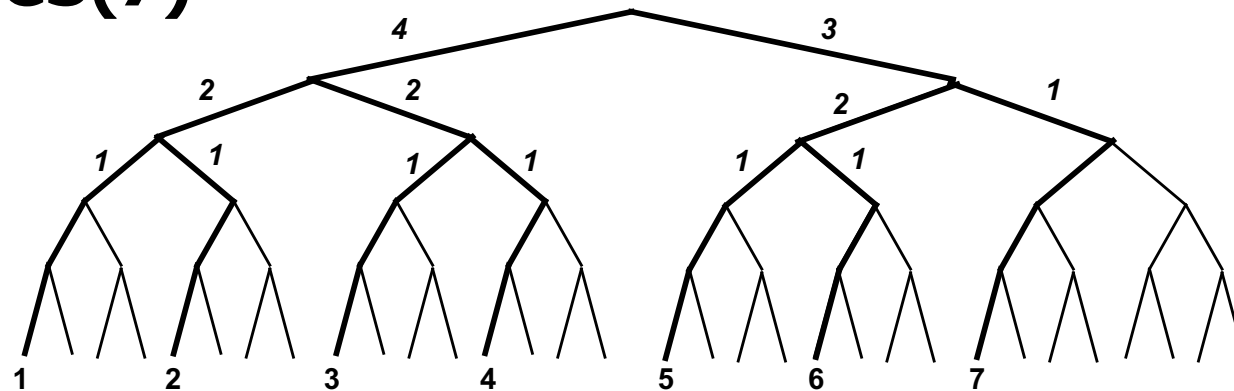
- credit is split among available alternatives
- unit credit means no alternatives (values)

*int divided
to ints*

1 = 1 větev

After failure, **increase credit by one** (restart).

Example: CS(7)



*nice, když mám
tušení, kam
do stranou
koukat*

Implementation:

$O(k)$

- in each node the (non-unit) credit is uniformly split among alternative sub-trees
- for a unit credit, just one alternative is tried



When solving real-life problems we frequently have some experience with “manual” solving of the problem.

Heuristics – a guide where to go

- they recommend a value for assignment (value ordering)
- frequently lead to a solution

But what to do when the heuristic is wrong?

- DFS takes care about the end of branches (leafs of tree)
- it repairs latest failures of the heuristic rather than earlier failures
- so it assumes that heuristic was right at the beginning of search

Observation1:

The number of wrong heuristic decisions is **low**.

Observation2:

Heuristics are usually **less reliable at the beginning** of search than at its end (more information and fewer choices are available there).

+ pruning of search space

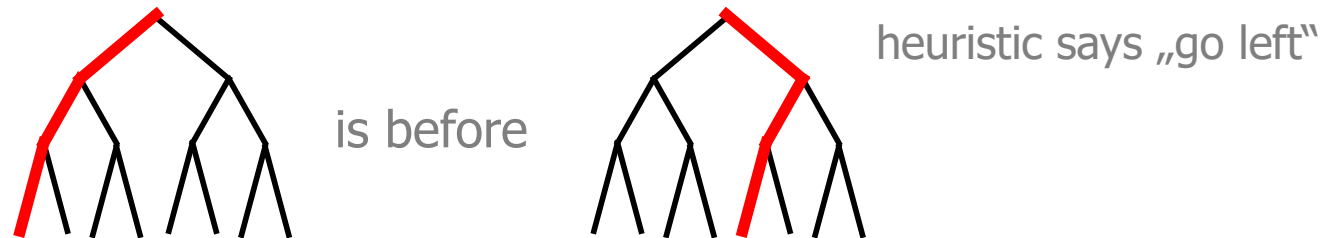
How to make search more efficient?

- Backtracking is “blind” with respect to heuristics.

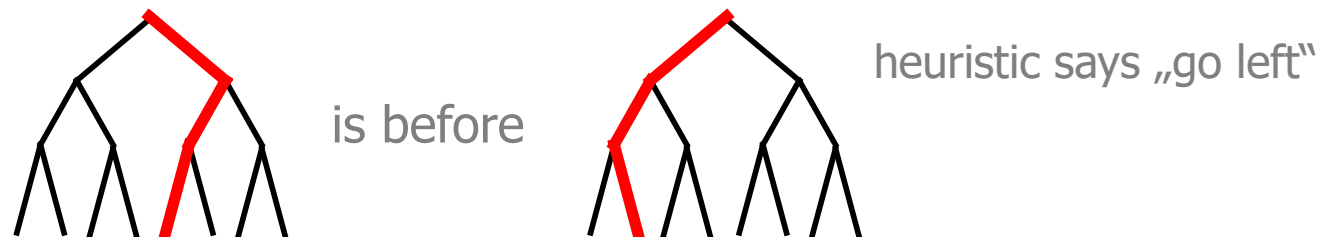
Discrepancy = violation of heuristic (different value is used)

Core principles of discrepancy search:

- we change the order of branches based on discrepancies
- explore first the branches with **less discrepancies**



- explore first the branches with **earlier discrepancies**



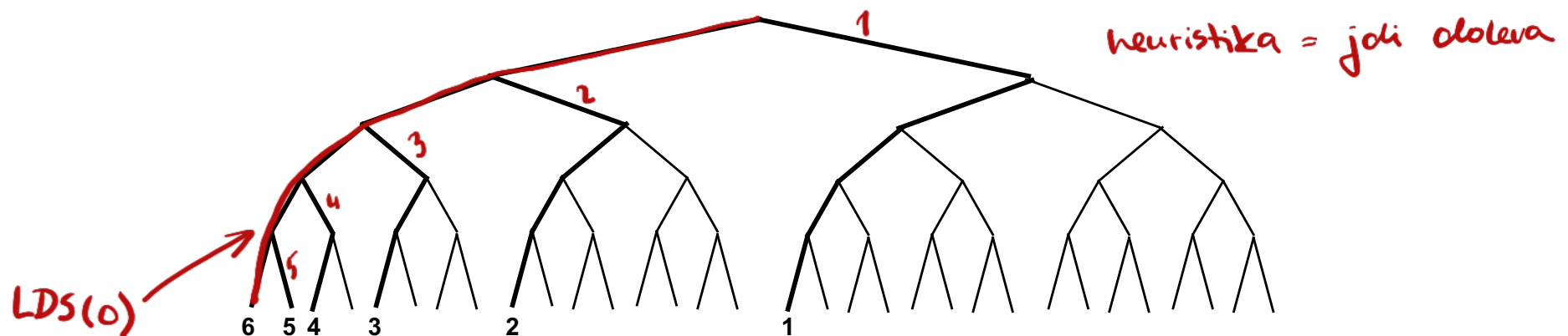
Limited number of discrepancies (cutoff)

- branches with less discrepancies are explored first

After failure **increase the number of allowed discrepancies** by one (restart).

- first, follow the heuristic
- then explore paths with at most one discrepancy

Example: LDS(1), *1 na cestu root-leaf*, heuristic suggests going to left



A note for **non-binary domains:**

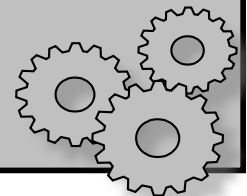
- non-heuristic values are assumed as **one discrepancy** (here)
- each other non-heuristic value means **increase of the number of discrepancies** (e.g. third value = two discrepancies)

```

procedure LDS-PROBE(Unlabelled, Labelled, Constraints, D)
  if Unlabelled = {} then return Labelled
  select X in Unlabelled
  ValuesX ← DX - {values inconsistent with Labelled using Constraints}
  if ValuesX = {} then return fail
  else select HV in ValuesX using heuristic
    if D > 0 then → discrepancy allowed
      for each value V from ValuesX - {HV} do
        R ← LDS-PROBE(Unlabelled - {X}, Labelled ∪ {X/V}, Constraints, D-1)
        if R ≠ fail then return R
      end for
    end if
  return LDS-PROBE(Unlabelled - {X}, Labelled ∪ {X/HV}, Constraints, D)
end if
end LDS-PROBE

procedure LDS(Variables, Constraints)
  for D = 0 to |Variables| do    % D determines the allowed number of discrepancies
    R ← LDS-PROBE(Variables, {}, Constraints, D)
    if R ≠ fail then return R
  end for
  return fail
end LDS

```

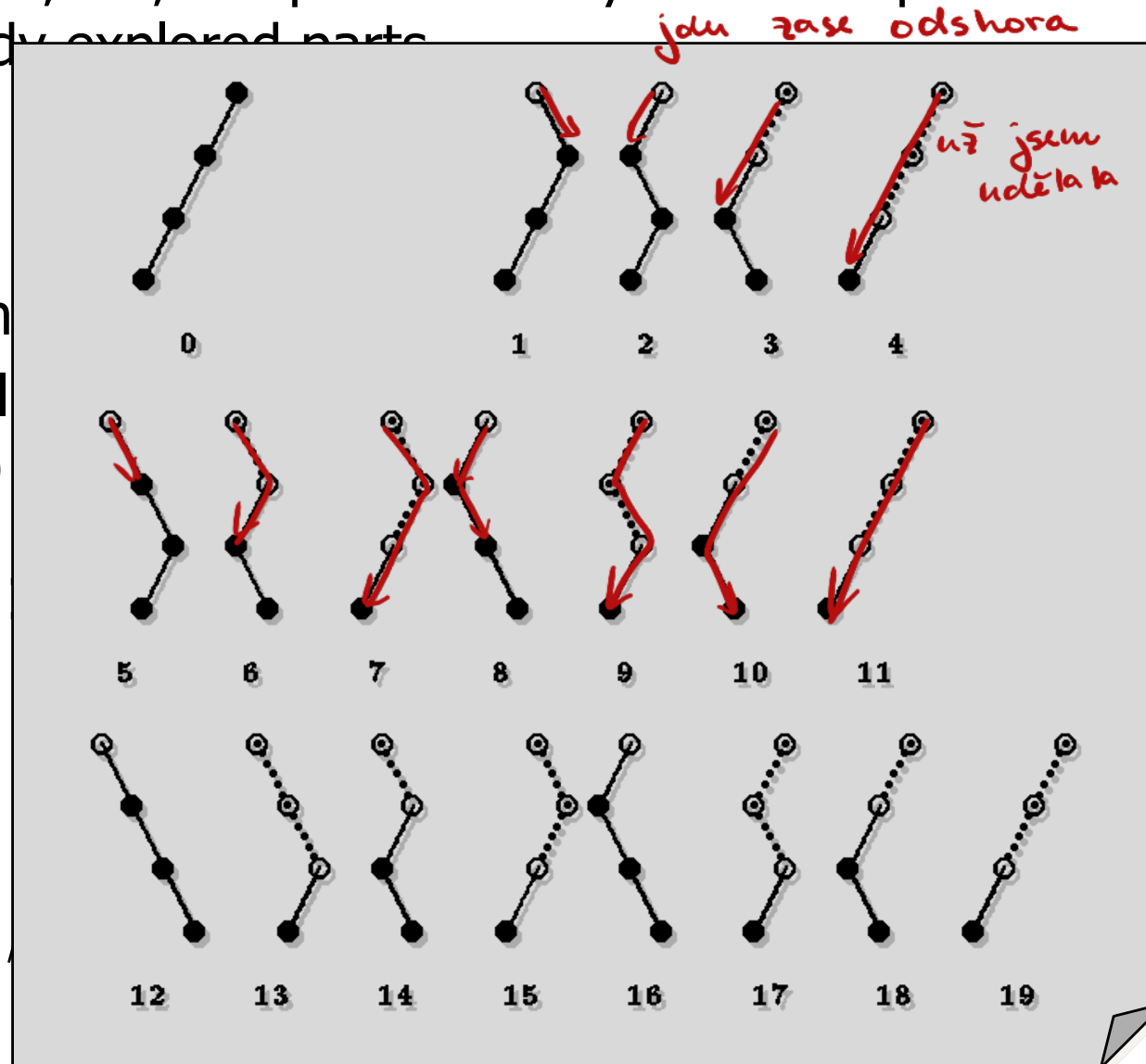


In each iteration LDS **explores branches from the previous iteration**, i.e., it repeats already done computation and returns to already explored parts

⇒ **ILDS:**

- a given – “bran
- After fail (restart)

Example:



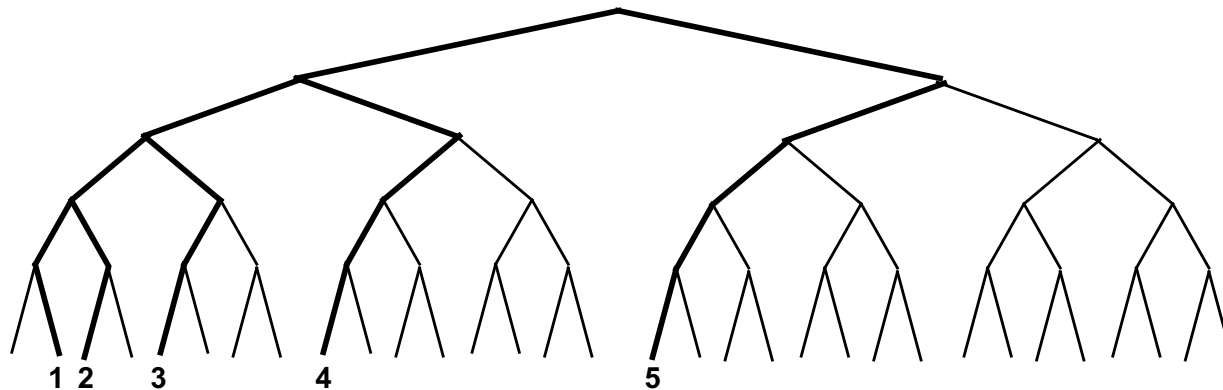
s by one

In each iteration LDS **explores branches from the previous iteration**, i.e., it repeats already done computation and returns to already explored parts.

⇒ **ILDS:**

- **a given number of discrepancies** (cutoff)
 - “branches with later discrepancies explored first”
- After failure **increase the number of discrepancies by one** (restart)

Example: ILDS(1), heuristic suggests going to left



```

procedure ILDS-PROBE(Unlabelled, Labelled, Constraints, D)
  if Unlabelled = {} then return Labelled
  select X in Unlabelled
  ValuesX ← DX - {values inconsistent with Labelled using Constraints}
  if ValuesX = {} then return fail
  else select HV in ValuesX using heuristic
    if D < |Unlabelled| then
      R ← ILDS-PROBE(Unlabelled - {X}, Labelled ∪ {X/HV}, Constraints, D)
      if R ≠ fail then return R
    if D > 0 then
      for each value V from ValuesX - {HV} do
        R ← ILDS-PROBE(Unlabelled - {X}, Labelled ∪ {X/V}, Constraints, D-1)
        if R ≠ fail then return R
      end for
    end if
  end ILDS-PROBE

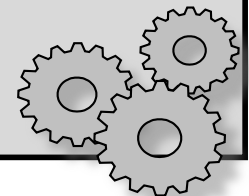
```

*Difference
from LDS*

```

procedure ILDS(Variables, Constraints)
  for D=0 to |Variables| do    % D determines the allowed number of discrepancies
    R ← ILDS-PROBE(Variables, {}, Constraints, D)
    if R ≠ fail then return R
  end for
  return fail
end LDS

```



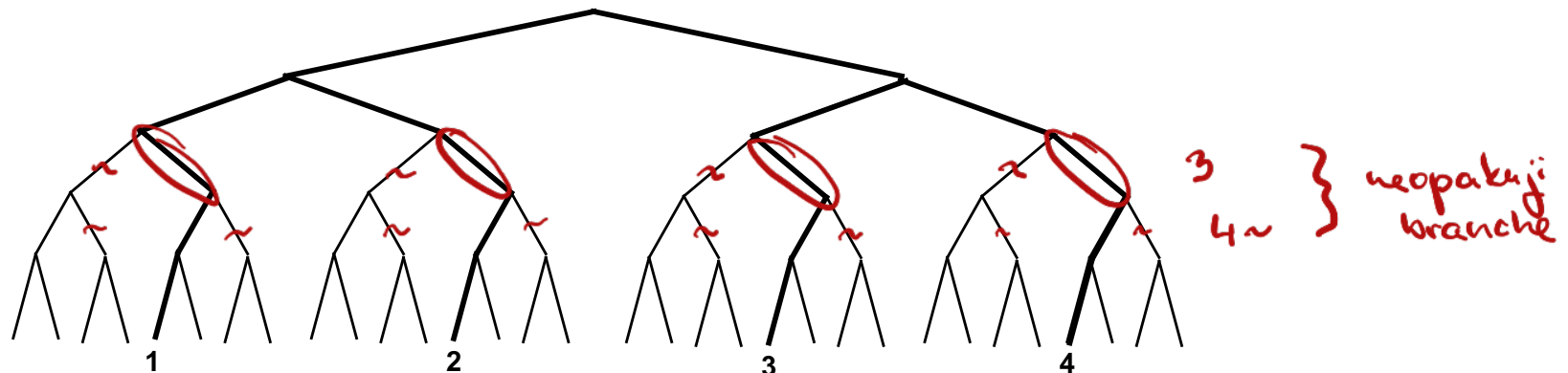
↳ Barták, Křiváček, Dolžanský

ILDS explores branches with later discrepancies first.

↪ **DDS:**

- **Discrepancies allowed to some depth** (cutoff)
 - at the limit depth, there must be discrepancy (so no branches from previous iterations are re-visited)
 - depth limit also restricts the number of discrepancies
 - branches with earlier discrepancies are tried first
- After failure **increase the depth limit by one** (restart)

Example: DDS(3), heuristic suggests going to left



So far we looked for any solution satisfying the constraints.

Frequently, we need to find an optimal solution, where solution quality is defined by some objective function.

Definition:

- **Constraint Satisfaction Optimisation Problem** (CSOP) consists of a CSP P and an objective function f mapping solutions of P to real numbers.
- A **solution to a CSOP** is a solution to P minimizing / maximizing the value of f .
- When solving CSOPs we need methods that can provide more than one solution.

The method **branch-and-bound** is a frequently used optimisation technique based on pruning branches where there is no optimal solution.

It uses a **heuristic function** h that estimates the value of objective function f .

- admissible heuristic for minimization satisfies $h(x) \leq f(x)$
[for maximization $f(x) \leq h(x)$]
- heuristic closer to f is better

We stop exploring the search branch when:

- there is **no solution** in the sub-tree
- there is **no optimal solution** in the sub-tree
 - $\text{Bound} \leq h(x)$, where Bound is the maximal value of f for an acceptable solution

How to obtain the Bound?

- for example the value of the solution found so far

Objective function is encoded in a constraint

we “optimize” the value v , where $v = f(x)$

- the first solution is found using no bound on v
- the next solutions must be better than the last solution found ($v < \text{Bound}$)
- repeat until no feasible solution is found

Algorithm **Branch & Bound**

procedure BB-Min(Variables, V , Constraints)

Bound $\leftarrow$ sup

NewSolution $\leftarrow$ fail

repeat

Solution $\leftarrow$ NewSolution

NewSolution $\leftarrow$ Solve(Variables, Constraints $\cup$ { $V < \text{Bound}$ })

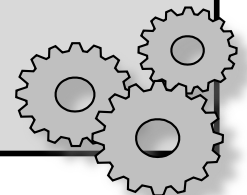
Bound $\leftarrow$ value of V in NewSolution (if any)

until NewSolution = fail

return Solution

end BB-Min

newly constraint



- Heuristic h is hidden in the **propagation of constraint** $v = f(x)$.
- Efficiency of search depends on:
 - **good heuristic** (good propagation through the objective constraint)
 - **good solution found early**
using an initial bound may help
- We can find the optimal solution fast
 - but the **proof of optimality takes time** (explore the rest of search tree)
- Frequently, we do not need optimal solution, good solution is enough
 - BB can stop after finding a good enough solution
- BB can be speeded up by using both upper and lower bounds

repeat

TempBound $\leftarrow$ ^{bounds} (UBound+LBound) / 2

NewSolution $\leftarrow$ Solve(Variables, Constraints $\cup$ $\{V \leq \text{TempBound}\}$)

if NewSolution=fail **then**

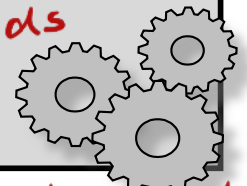
LBound $\leftarrow$ TempBound+1

else

UBound $\leftarrow$ TempBound

until LBound = UBound $\rightarrow$ optimum

ALL musím
znát
ty bounds



výhoda = logaritmičeský rychlejší než jen B&B + vím jak daleko jsem od optima



© 2013 Roman Barták

Department of Theoretical Computer Science and Mathematical Logic

bartak@ktiml.mff.cuni.cz